DOCUMENTAÇÃO TÉCNICA COMPLETA

Sistema de Gestão - Oficina AutoCerta

Manual do sistema, arquitetura, banco de dados, API & Segurança

Versão 1.0 | Outubro de 2026

1. Apresentação

O **Oficina AutoCerta** é uma aplicação web destinada a apoiar a organização de uma oficina mecânica. O sistema reúne em um único ambiente o acesso de usuários, o cadastro e a consulta de clientes, o registro de veículos e o controle de agendamentos de serviços.

A aplicação utiliza páginas HTML, folhas de estilo CSS e JavaScript no navegador. O servidor é desenvolvido em Node.js com Express, e os dados são armazenados em um banco SQLite local. As telas conversam com o servidor por meio de uma API HTTP que retorna dados em formato JSON.

> **1.1 Objetivo Geral:** Desenvolver um sistema web simples para centralizar informações de clientes, veículos e serviços agendados, reduzindo a dependência de anotações manuais e facilitando a consulta das informações da oficina.

1.2 Objetivos Específicos

- Autenticar usuários por login e senha com retenção segura de sessão.
- Permitir consultar, pesquisar, cadastrar e editar dados de clientes.
- Cadastrar veículos e vinculá-los ao respectivo proprietário/cliente.
- Cadastrar agendamentos de serviços e alterar a data de um agendamento existente.
- Validar dados rigidamente no servidor antes de gravá-los no banco SQLite.
- Proteger a sessão de acesso e armazenar senhas e CPFs com medidas de criptografia.

1.3 Escopo e Limites

O escopo implementado inclui login/logout, consulta do usuário autenticado, clientes, veículos e agendamentos. No código analisado, não há rota para excluir clientes, veículos ou agendamentos; também não há gestão de permissões por perfil, relatórios financeiros, estoque de peças ou envio automático de lembretes. Essas funções representam melhorias futuras.

2. Tecnologias Utilizadas

Tecnologia	Uso e Responsabilidade no Sistema
HTML5	Estrutura semântica das páginas e formulários web.
CSS3	Estilização, layout responsivo, tabelas, cartões e janelas modais.
JavaScript (ES6+)	Manipulação do DOM, manipulação de eventos e chamadas à API via Fetch.
Node.js 22.13+	Ambiente de execução backend e suporte ao módulo nativo `node:sqlite`.
Express 5	Servidor HTTP, roteamento REST da API e distribuição de arquivos estáticos.
express-session	Gerenciamento de sessões de usuários autenticados via cookies.
bcryptjs	Criptografia hash e comparação segura de senhas.
SQLite 3 / node:sqlite	Banco de dados relacional armazenado localmente em arquivo físico.

3. Requisitos para Execução e Instalação

Para executar o sistema localmente, é necessário garantir que o ambiente atenda aos requisitos mínimos de software e dependências especificados.

3.1 Requisitos Mínimos & Credenciais de Demonstração

Requisito	Especificação
Sistema Operacional	Windows 10/11, Linux ou macOS.
Ambiente Node.js	Versão 22.13 ou superior (obrigatório para `node:sqlite`).
Navegador Web	Google Chrome, Mozilla Firefox, Microsoft Edge ou Safari atualizados.

Login	Senha de Demonstração	Identificação / Perfil Exibido
`admin`	`admin123`	Administrador Geral
`ana.mecanica`	`ana123`	Ana Beatriz (Mecânica Chefe)
`carlos.recepcao`	`carlos123`	Carlos Lima (Recepção)

3.2 Instruções de Instalação e Execução

- Extraia o arquivo ZIP do projeto para uma pasta local de sua preferência.
- Abra o terminal dentro da pasta `sistema` (onde se encontra o arquivo `package.json`).
- Confirme a versão do Node executando `node -v`. Ela deve ser igual ou superior a 22.13.
- Execute o comando `npm install` para baixar as dependências do projeto.
- Inicie o servidor com o comando `npm start`.
- Acesse a aplicação no navegador pelo endereço `http://localhost:3000`.
- *Nota:* Na primeira execução, o banco `db/oficina_db.sqlite` é criado a partir do script SQL base.

4. Organização Estrutural dos Arquivos

Caminho / Arquivo	Descrição da Responsabilidade
`server.js`	Servidor principal Express, rotas da API, middlewares e gerenciamento de sessão.
`seguranca.js`	Módulo de criptografia AES-256-GCM para cifragem e decifragem de CPFs.
`package.json`	Definições de dependências, scripts de inicialização e versões requeridas.
`db/conexao.js`	Conectores SQLite, inicialização de banco e ativação de Foreign Keys.
`db/oficina_db.sql`	Script SQL com DDL das tabelas e DML dos dados iniciais de testes.
`db/gerar_sql.js`	Utilitário para recalcular hashes bcrypt e criptografar CPFs do script base.
`public/login.html`	Interface web para autenticação de usuários.
`public/principal.html`	Painel inicial de navegação entre módulos do sistema.
`public/clientes.html`	Módulo visual para cadastro, busca e listagem de clientes.
`public/veiculos.html`	Módulo visual para controle de frota e vínculo com proprietários.
`public/agendamentos.html`	Módulo visual para controle de serviços e datas de atendimento.

5. Descrição Funcional dos Módulos

5.1 Login & Autenticação: O usuário informa login e senha no formulário web. O navegador envia uma requisição `POST /api/login`. O servidor valida a presença dos campos, busca a conta no banco de dados e valida o hash bcrypt. Se bem-sucedido, inicializa a sessão HTTP.

5.2 Painel Principal: Atua como hub central apresentando o usuário logado e atalhos para os módulos operacionais, além do botão de encerramento de sessão (Logout).

5.3 Gestão de Clientes: Permite busca em tempo real por nome, CPF, telefone ou e-mail. Exige nome e CPF válido com 11 dígitos no cadastro, aplicando criptografia no CPF antes da gravação no banco de dados.

5.4 Gestão de Veículos: Realiza normalização de placas para caixa alta sem caracteres especiais. Suporta padrão antigo e Mercosul, exigindo vínculo com um cliente existente e impedindo a duplicação de placas no sistema.

5.5 Agendamentos: Controla serviços e prazos vinculados a veículos. Permite edição rápida da data de atendimento e validações de integridade temporal (datas válidas no formato AAAA-MM-DD).

6. Arquitetura do Sistema e Modelo de Dados

O sistema adota a arquitetura em três camadas (3-Tier): Apresentação (Frontend), Aplicação/Negócio (Backend Node/Express) e Persistência (SQLite).

```
      [ Navegador Web ] ──(HTTP/JSON)──> [ API Express / Node.js ] ──(SQL)──> [ SQLite
Database ] (HTML5/CSS3/JS) <──(Respostas)── (Validações / Regras) <──(Dados)──
(oficina_db.sqlite)
```

7.1 Estrutura de Tabelas (Dicionário de Dados)

Tabela	Campos Principais	Regras e Restrições
`usuarios`	id (PK), nome, login (UNIQUE), senha	Armazena credenciais de acesso. Senhas em hash bcrypt.
`clientes`	id (PK), nome, cpf, telefone, email	Dados dos clientes. CPF gravado com criptografia AES-256.
`veiculos`	id (PK), placa (UNIQUE), modelo, tipo, marca, cliente_id (FK)	Veículos cadastrados. Vinculado obrigatoriamente a um cliente.
`agendamentos`	id (PK), veiculo_id (FK), data_agendamento, servico	Registro de ordens/agendamentos atrelados a um veículo.

7.2 Diagrama de Entidade e Relacionamento (DER Textual)

```
      +-------------------+ 1 : N +-------------------+ 1 : N +-------------------+ |
CLIENTES | ----------------> | VEICULOS | ----------------> | AGENDAMENTOS | +---------------
----+ +-------------------+ +-------------------+ | PK: id | | PK: id | | PK: id | | nome | |
placa (UNIQUE)| | FK: veiculo_id | | cpf (Encrypted| | modelo | | data_agendam. | | telefone
| | tipo, marca | | servico | | email | | FK: cliente_id | +-------------------+ +-----------
--------+ +-------------------+ +-------------------+ | USUARIOS | (Entidade independente
para autenticação de operadores) +-------------------+ | PK: id | login (UNIQUE) | senha
(bcrypt hash) | nome | +-------------------+
```

8. ESPECIFICAÇÃO DA API REST

Todas as rotas retornam e recebem payloads no formato JSON. Exceto as rotas de login/logout, todos os endpoints exigem sessão ativa.

Método & Rota	Acesso	Descrição e Regras de Validação
`POST /api/login`	Público	Autentica o usuário. Retorna 400 (campos ausentes) ou 401 (senha/login inválidos).
`POST /api/logout`	Sessão	Destrói a sessão do usuário e invalida o cookie. Retorna `{ok: true}`.
`GET /api/me`	Privado	Retorna a identificação do usuário conectado. 401 para não autenticados.
`GET /api/clientes`	Privado	Lista clientes ordenados por nome. Aceita parâmetro `?busca=`.
`POST /api/clientes`	Privado	Cadastra cliente. Valida nome, CPF (11 dígitos, único) e e-mail. Retorna 201.
`PUT /api/clientes/:id`	Privado	Atualiza dados do cliente. Retorna 404 se não encontrado.
`GET /api/veiculos`	Privado	Retorna todos os veículos acompanhados do nome do cliente proprietário.
`POST /api/veiculos`	Privado	Cadastra veículo. Normaliza placa, verifica duplicidade e cliente_id. Retorna 201.
`GET /api/agendamentos`	Privado	Lista agendamentos com dados acoplados de veículo e cliente por data.
`POST /api/agendamentos`	Privado	Cria agendamento. Exige veículo válido, data futura/válida e descrição. Retorna 201.
`PUT /api/agendamentos/:id`	Privado	Altera exclusivamente a data de um agendamento pré-existente.

9. Segurança, Criptografia e Privacidade

O sistema adota padrões defensivos essenciais para proteção de dados e integridade dos registros:

- **Hashes de Senha:** As senhas jamais são salvas em texto limpo. Utiliza-se a biblioteca `bcryptjs` para geração de sal e hash irreversível.
- **Proteção de Dados Pessoais (LGPD):** O CPF é classificado como dado pessoal. O sistema realiza cifragem simétrica com o algoritmo `AES-256-GCM` através do módulo `crypto` do Node.js antes de persistir no banco.
- **Proteção de Sessão:** Cookies de sessão são configurados com a flag `httpOnly`, impedindo acesso via scripts client-side (XSS).
- **Prevenção contra SQL Injection:** Todas as rotas usam consultas parametrizadas com placeholders `?` no SQLite.

> **Recomendações para Produção:** Em implantações reais, é mandatório mover as chaves de criptografia e segredos de sessão para variáveis de ambiente (`.env`), habilitar a flag `secure` nos cookies para tráfego HTTPS e implementar proteção contra ataques de força bruta (Rate Limiting).

10. Roteiro de Testes Recomendado

ID	Caso de Teste	Procedimento / Entrada	Resultado Esperado
T01	Navegação inicial	Acessar `http://localhost:3000`	Redirecionamento automático para a tela de login.
T02	Login válido	Entrar com `admin` / `admin123`	Autenticação aceita e navegação ao painel principal.
T03	Login inválido	Entrar com senha incorreta	Exibição de mensagem de erro e rejeição de acesso.
T04	Acesso sem sessão	Tentar abrir `/api/clientes` sem logar	Resposta HTTP status 401 Unauthorized.
T05	Cadastro de cliente sem nome	Enviar formulário de cliente com nome em branco	Recusa no servidor com mensagem de validação.
T06	CPF Inválido/Incompleto	Tentar cadastrar CPF com menos de 11 dígitos	Validação bloqueia envio e aciona alerta visual.
T07	CPF Duplicado	Cadastrar cliente com CPF já existente	Servidor recusa a inserção devido à restrição de unicidade.
T08	Placa Inválida	Cadastrar veículo com placa fora do padrão	Bloqueio de cadastro por formato incorreto.
T09	Agendamento em data inválida	Tentar agendar para 31/02/2026	Validador de data recusa a requisição HTTP.
T10	Alteração de Data	Mudar a data de um agendamento na lista	Persistência imediata e atualização no banco SQLite.
T11	Encerramento de Sessão	Clicar em "Sair" e tentar acessar rota interna	Sessão destruída e retorno obrigatório à tela de login.
T12	Persistência de Dados	Reiniciar o servidor Node.js	Todos os dados adicionados permanecem acessíveis.

11. Resolução de Problemas (Troubleshooting)

- **Comando `node` não reconhecido:** O Node.js não está no PATH do sistema. Reinstale a versão 22.13+.
- **Porta 3000 já em uso:** Encerre o processo em execução ou altere a constante `PORT` no arquivo `server.js`.
- **Módulo `node:sqlite` ausente:** A versão do Node.js em execução é inferior à 22.13.0. Atualize o ambiente.
- **Alterações no arquivo `oficina_db.sql` não aplicadas:** O script só roda na criação do banco. Se `oficina_db.sqlite` já existir, apague-o para recriar o banco do zero.

12. Conclusão

O sistema **Oficina AutoCerta** atende satisfatoriamente aos requisitos de gestão operacional de oficinas mecânicas de pequeno e médio porte. Sua arquitetura modular e o uso de SQLite com criptografia AES-256 e hashes bcrypt garantem integridade, facilidade de implantação local e conformidade basilar com boas práticas de desenvolvimento web e proteção de dados.